



Ejercicio 02: Modelos Declarativos

Objetivo

Practicar la creación de modelos SQLAlchemy con diferentes tipos de datos, restricciones y valores por defecto.

Duración estimada: 40 minutos



Pasos

Paso 1: Setup Inicial

Configuración base que usaremos en todo el ejercicio:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

engine = create_engine("sqlite:///./models_test.db", echo=True)

class Base(DeclarativeBase):
    pass
```

Abre `starter/main.py` y descomenta la sección del Paso 1.

Paso 2: Modelo con Tipos Básicos

Tipos fundamentales en SQLAlchemy:

```
class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    price: Mapped[float]
    in_stock: Mapped[bool] = mapped_column(default=True)
```

- `Mapped[int]` → INTEGER en SQL
- `Mapped[str]` requiere `String(length)`
- `Mapped[float]` → FLOAT en SQL
- `Mapped[bool]` → BOOLEAN en SQL

Descomenta la sección del Paso 2.

Paso 3: Campos Opcionales (Nullable)

Usar `| None` para campos que pueden ser NULL:

```
class Customer(Base):
    __tablename__ = "customers"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str | None] = mapped_column(String(255)) # Nullable
    phone: Mapped[str | None] = mapped_column(String(20)) # Nullable
```

Descomenta la sección del Paso 3.

Paso 4: Valores por Defecto

Python defaults vs Server defaults:

```
from datetime import datetime

class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    status: Mapped[str] = mapped_column(String(20), default="pending")
    total: Mapped[float] = mapped_column(default=0.0)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

- `default=value` → Python genera el valor antes de INSERT
- `server_default="value"` → La DB genera el valor

Descomenta la sección del Paso 4.

Paso 5: Restricciones (Constraints)

Asegurar integridad de datos:

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(50), unique=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    age: Mapped[int | None] = mapped_column(CheckConstraint("age >= 0"))
```

- `unique=True` → No permite duplicados
- `index=True` → Crea índice para búsquedas rápidas
- `CheckConstraint` → Validación a nivel de DB

Descomenta la sección del Paso 5.

Paso 6: Crear Tablas y Probar

Crear todas las tablas y probar con datos:

```
Base.metadata.create_all(bind=engine)

with SessionLocal() as session:
    product = Product(name="Laptop", price=999.99)
    session.add(product)
    session.commit()
```

Descomenta la sección del Paso 6.

 Ejecutar

```
uv run python starter/main.py
```

Salida esperada:

```
=== Paso 1: Setup completado ===
=== Paso 2: Modelo Product definido ===
=== Paso 3: Modelo Customer definido ===
=== Paso 4: Modelo Order definido ===
=== Paso 5: Modelo User definido ===
=== Paso 6: Probando modelos ===
Product: Product(id=1, name='Laptop', price=999.99, in_stock=True)
Customer: Customer(id=1, name='John', email=None, phone=None)
Order: Order(id=1, status='pending', total=0.0, created_at=...)
User: User(id=1, username='john_doe', email='john@example.com', age=25)
```

✓ Verificación

- ☐ Todos los modelos se crearon sin errores
- ☐ El archivo `models_test.db` existe
- ☐ Los defaults se aplican correctamente
- ☐ Los campos nullable aceptan None

🧠 Tabla de Tipos

Python	SQLAlchemy	SQL
int	Integer	INTEGER
str	String(n)	VARCHAR(n)
float	Float	FLOAT
bool	Boolean	BOOLEAN
datetime	DateTime	DATETIME
T None	nullable	NULL allowed